

RELATÓRIO PRÁTICO / PEDIDOS VELOZ

LOJA VELOZ / CLOUD DEVOPS

Entrega contínua de uma plataforma de pedidos em microsserviços

Parte prática - do Docker Compose ao Kubernetes com CI/CD

Aluno: Antonio Silva | Curso: Análise e Desenvolvimento de Sistemas

Instituição: Unifecaf | Tutor: Fernando Melo

Vídeo: link será inserido após a gravação

Visão geral da solução

O repositório entrega um MVP executável da Loja Veloz. O Gateway concentra a API pública, Pedidos controla estado e histórico, Estoque reserva e compensa quantidades, e Pagamentos simula uma autorização externa. PostgreSQL mantém um banco por domínio e RabbitMQ transporta eventos duráveis. O desenho permite demonstrar confirmação, falta de estoque e recusa de pagamento sem usar dados reais.

Etapa	Responsável	Resultado
1. Entrada	Gateway / Pedidos	202 e pedido AGUARDANDO_ESTOQUE
2. Reserva	Estoque	Reserva total ou cancelamento
3. Cobrança	Pagamentos	Aprovação ou recusa de pagamento
4. Fechamento	Pedidos / Estoque	Confirmação ou devolução

Contrato assíncrono

Todo evento carrega `id\_evento`, `tipo`, `versao`, `ocorrido\_em`, `correlacao\_id` e `dados`. A chave de roteamento é o tipo, como `pedido.criado` ou `pagamento.recusado`. O `correlacao\_id` é o UUID do pedido e também liga logs e traces. Os consumidores confirmam a mensagem somente depois do commit local, e em falha o RabbitMQ recoloca a entrega na fila.

A outbox usa bloqueio temporário e contador de tentativas. Se a publicação não for confirmada, o registro volta a ficar elegível. Já `eventos\_processados` impede efeitos repetidos. Esse conjunto não promete entrega exatamente uma vez, mas garante resultado de negócio idempotente, solução mais realista para filas distribuídas.

RELATÓRIO PRÁTICO / PEDIDOS VELOZ

Ambiente local e containerização

O arquivo ``docker-compose.yml`` sobe toda a plataforma com ``docker compose up --build -d``. Há três redes: aplicação, dados e observabilidade. A rede de dados é interna e não publica PostgreSQL. Volumes preservam bancos, filas e dashboards. Healthchecks coordenam a ordem de inicialização sem depender de esperas fixas. ``.env.example`` documenta parâmetros, enquanto ``.env`` fica ignorado pelo Git.

Cada Dockerfile usa dois estágios. O primeiro instala dependências em um ambiente virtual, e o segundo recebe somente runtime, ambiente e código. O processo usa UID 10001, não cria home, grava temporários em ``/tmp`` e não possui capabilities. No Compose, os serviços ficam com root filesystem somente leitura, ``no-new-privileges`` e volume temporário limitado. Versões das imagens-base e bibliotecas estão fixadas para evitar builds imprevisíveis.

Execução local

Depois de copiar ``.env.example`` para ``.env``, o Gateway fica em ``localhost:8000`` e a documentação Swagger em ``/docs``. Os três JSON em ``exemplos/`` cobrem os caminhos principais. Um token iniciado por ``falha_`` força recusa, e esse valor é fictício e nunca deve ser substituído por cartão real. ``scripts/teste_smoke.py`` cria um pedido aprovado e consulta o estado até a confirmação.

Componente	Endereço	Finalidade
Swagger	localhost:8000/docs	Criar e consultar pedidos
Grafana	localhost:3000	Painel e Explore
RabbitMQ	localhost:15672	Filas e consumidores
Prometheus	localhost:9090	Métricas e alertas

Para encerrar sem perder dados usa-se ``docker compose down``, e com ``--volumes`` o laboratório volta ao estado inicial. O README registra ambos para evitar exclusão acidental.

Kubernetes: configuração entregue

Kustomize separa uma base reutilizável e um overlay que representa o ambiente de produção. Os manifests descrevem quatro Deployments com duas réplicas, requests, limits, probes e rolling update com `maxUnavailable: 0` e `maxSurge: 1`. Services mantêm endereços internos e apenas o Gateway usa LoadBalancer. PostgreSQL e RabbitMQ aparecem como StatefulSets com PVC. Essa configuração serve como referência de implantação e não foi aplicada em um cluster externo.

Readiness testa dependências e retira um pod do balanceamento enquanto banco ou fila não respondem. Liveness verifica apenas se o processo continua vivo, evitando reinícios em cascata por uma dependência externa. PDB mantém ao menos uma réplica do Gateway e de Pedidos em interrupções voluntárias. HPA varia entre duas e seis réplicas a 70% de CPU, apoiado por requests explícitos e Metrics Server.

Configuração e segurança

ConfigMap contém URLs internas, ambiente e nível de log. O Secret esperado guarda usuários, senhas e URLs completas dos bancos, criado pelo pipeline diretamente no namespace. `segredos.exemplo.yaml` é apenas referência. O namespace aplica Pod Security `restricted`, e as aplicações executam sem root, com seccomp RuntimeDefault, capabilities removidas, sem elevação e com filesystem somente leitura. NetworkPolicies começam negando tudo e liberam DNS e os fluxos necessários.

Controle	Aplicação no projeto
Probes	Saúde separada de prontidão
Rolling update	Troca gradual com rollback no pipeline
HPA	2 a 6 réplicas por CPU média
Pod Security	Perfil restricted no namespace
NetworkPolicy	Comunicação liberada por papel e porta

Istio ficou fora do MVP. OpenTelemetry já cobre rastreabilidade, e um mesh será útil quando mTLS uniforme e canary com divisão de tráfego justificarem o custo operacional.

CI/CD, imagens e GitHub

A integração contínua roda em pull requests e em `main`. O primeiro job executa Ruff, mypy nos contratos e pytest com cobertura mínima. O segundo valida Compose, renderiza Kustomize, usa kubeconform e executa `terraform validate`. O terceiro procura secrets e vulnerabilidades com Gitleaks e Trivy. O último sobe os componentes necessários no Compose e executa o pedido smoke de ponta a ponta.

O workflow de entrega reage a tags `v\*` ou acionamento manual. Uma matriz constrói as quatro imagens e publica no GHCR com versão semântica e tag do SHA. O token nativo do GitHub recebe apenas permissões de leitura do repositório e escrita de packages. Depois da primeira publicação, os pacotes devem ficar públicos para que o cluster faça pull sem credencial adicional.

Deploy protegido e rollback

O deploy permanece opcional e desabilitado por padrão. Ele só é iniciado com `HABILITAR\_DEPLOY=true` e um cluster previamente configurado. O environment `producao` exige aprovação, kubeconfig e senhas. Sem esses recursos, o pipeline mantém apenas as validações de código, infraestrutura e segurança.

Infraestrutura como código

O diretório Terraform contém uma proposta opcional de infraestrutura, com variáveis, outputs e exemplo de backend remoto. Nenhum recurso externo foi criado. O projeto não executa `terraform apply` automaticamente, pois essa ação pode gerar cobrança e exige uma conta externa. No MVP, a infraestrutura como código funciona como modelo documentado e pode ser validada estaticamente pelo GitHub Actions.

Gatilho	Saída
Pull request	Testes, manifests, IaC e scans
Tag v1.0.0	Quatro imagens versionadas no GHCR
Deploy habilitado	Rolling update aprovado no ambiente

Observabilidade e resiliência

O middleware mede quantidade e duração HTTP por serviço, rota, método e status. Contadores de negócio acompanham pedidos, estoque e pagamentos. Prometheus coleta os endpoints e as métricas do RabbitMQ, e duas regras alertam indisponibilidade e taxa de erro acima de 5%. O dashboard provisionado mostra requisições por segundo, p95, respostas recentes e logs.

Logs saem em JSON com horário, nível, logger e mensagem. Quando existe span ativo, `trace\_id` e `span\_id` entram automaticamente. Grafana Alloy descobre contêineres Docker e envia os streams para Loki. Instrumentação FastAPI e HTTPX cria spans, e publicação e consumo do RabbitMQ têm spans manuais e propagam W3C Trace Context. O Collector envia os traces ao Tempo, onde uma operação pode ser seguida do Gateway ao pagamento.

Validações executadas

Cenário	Critério de aceite
Aprovado	CONFIRMADO, baixa única e reserva zerada
Sem estoque	CANCELADO antes do pagamento
Pagamento não autorizado	CANCELADO e quantidade devolvida
Evento duplicado	Nenhum segundo movimento
Outbox bloqueada	Nova tentativa após o prazo
Contratos	Entrada inválida rejeitada pelo Pydantic

Na máquina de elaboração, oito testes de regra passaram, com 90% de cobertura nos módulos centrais. Ruff e mypy não apontaram problemas, e 23 arquivos YAML foram carregados com sucesso. O Docker Compose foi executado com os serviços saudáveis. Também foram comprovados os cenários de pedido confirmado, estoque insuficiente e pagamento não autorizado, com devolução do estoque. As validações de kubectl e Terraform permanecem automatizadas no GitHub Actions.

Riscos remanescentes

O MVP ainda precisa de DLQ, painel de reprocessamento, migrações de banco, backup, TLS interno e teste de carga antes de uma campanha real. Consumidores deveriam escalar pelo tamanho da fila com KEDA. Esses itens foram registrados como evolução para não disfarçar limitações do laboratório como garantias de produção.

RELATÓRIO PRÁTICO / PEDIDOS VELOZ

Estado atual do projeto

A plataforma foi executada com Docker Compose e os serviços permaneceram saudáveis. Os cenários de pedido confirmado, estoque insuficiente e pagamento não autorizado foram validados junto com o painel de observabilidade. O repositório usa a branch `main` e o GitHub Actions para verificar código, infraestrutura e segurança.

A branch principal recebe mudanças revisadas por pull request e checks. O deploy externo permanece desabilitado sem um cluster configurado. Kubernetes e Terraform ficam como opções de infraestrutura, enquanto o ambiente operacional do MVP usa Docker Compose local.

Área	Estado
Repositório	README, Actions e código versionado
Execução	Compose saudável e três cenários reproduzíveis
Documentação	Relatórios teórico e prático
Observabilidade	Métricas, logs e traces correlacionados
Segurança	Sem .env, kubeconfig, state ou dado real

Conclusão

A proposta reduz variação no desenvolvimento, transforma a publicação em processo auditável e fornece sinais para investigar falhas. Compose mantém o ambiente executável local, GitHub Actions automatiza testes e imagens, os manifests Kubernetes documentam disponibilidade e escala, outbox e idempotência protegem o fluxo, e métricas, logs e traces devolvem contexto à operação. O projeto atende ao recorte do MVP sem afirmar que houve implantação em nuvem.

Referências essenciais

DOCKER. Compose e multi-stage builds. <https://docs.docker.com/>
KUBERNETES. Probes, HPA, Secrets e Security. <https://kubernetes.io/docs/>
GITHUB. Actions e publicação no GHCR. <https://docs.github.com/actions/>
OPENTELEMETRY. Demo e instrumentação. <https://opentelemetry.io/docs/demo/>
HASHICORP. Terraform. <https://developer.hashicorp.com/terraform/docs>
Acesso às fontes: 15 ago. 2026.